HDOC Day-59 Activity

Longest Substring Without Repeating Characters Using Pointers

Student Name	Dhruv Pandey
SAP ID	590037660
Batch	15
Course / Activity	HDOC Day-59

Aim

To write a C program that finds the length of the longest contiguous substring containing no repeated characters. The string is passed to **longest_unique_substring()** using its address and accessed using pointers.

Problem Requirements

- Read a string **s** from the user.
- Pass the string to **longest_unique_substring()** using its address.
- Find the longest substring containing no repeated characters.
- Display the length of the longest such substring.
- Treat uppercase and lowercase characters as different characters.
- Display 0 if the input string is empty.

Algorithm

1. Read the string into a character array.
2. Pass its address to **longest_unique_substring(s)**.
3. If the first character is the null character, print 0.
4. For every possible starting position i, begin a new substring.
5. Move j from i toward the end of the string.
6. Before including s[j], search from i to j-1. If the same character already exists, stop extending that substring.
7. Otherwise, calculate its current length and update the maximum length.
8. Display the maximum length.

C Program

```
#include <stdio.h>
#include <string.h>

void longest_unique_substring(char *s)
{
    if (*s == '\0')
    {
        printf("0");
        return;
    }

    int n = strlen(s);
    int max_length = 0;

    for (int i = 0; i < n; i++)
    {
        for (int j = i; j < n; j++)
        {
            int repeated = 0;

            for (int k = i; k < j; k++)
            {
                if (*(s + k) == *(s + j))
                {
                    repeated = 1;
                    break;
                }
            }

            if (repeated == 1)
            {
                break;
            }

            int length = j - i + 1;

            if (length > max_length)
            {
                max_length = length;
            }
        }
    }

    printf("Length of longest substring: %d", max_length);
}

int main()
{
    char s[1000];

    printf("Enter the string: ");
    fgets(s, sizeof(s), stdin);

    s[strcspn(s, "\n")] = '\0';

    longest_unique_substring(s);

    return 0;
}
```

Pointer Usage

longest_unique_substring(s) passes the address of the first character. The parameter **char *s** receives this address. The expression ***(s + j)** is equivalent to **s[j]**.

Example 1 Dry Run

Start	Characters accepted before repeat	Length
a	a b c (next a repeats)	3
b	b c a (next b repeats)	3
c	c a b (next c repeats)	3
a	a b (next b repeats)	2
b	b (next b repeats)	1

For **abcabcbb**, the longest unique substring length is **3**. One such substring is **abc**.

Test Cases / Expected Results

Case	Input String	Expected Output
Example 1	abcabcbb	Length of longest substring: 3
Example 2	bbbbb	Length of longest substring: 1
Example 3	pwwkew	Length of longest substring: 3
Example 4	Empty string	0

Example 2 Explanation

In **bbbbb**, adding a second b immediately creates a repetition. Therefore, the longest substring without repeated characters contains only one character and its length is **1**.

Example 3 Explanation

For **pwwkew**, substrings such as **wke** contain three distinct characters. Extending further introduces another w, so the maximum length is **3**.

Case Sensitivity

Character comparison in C is case-sensitive. Therefore, uppercase and lowercase letters such as **A** and **a** are treated as different characters, as required.

Empty String

If the first character is the null character **'\0'**, the string contains no characters. The function immediately displays **0**.

Complexity

Time Complexity: $O(n^3)$ in the worst case for this straightforward nested-loop approach, because each candidate extension may scan earlier characters for repetition.

Space Complexity: O(1) extra space apart from the input string.

Result

The program successfully finds the length of the longest substring containing no repeated characters using pointer-based string access. It treats uppercase and lowercase letters separately and displays 0 for an empty string.

Execution Example

```
Enter the string: abcabcbb
Length of longest substring: 3
```